



Relatório 03 | Organização de computadores I - INE5411

Alunos: Pedro Henrique Gimenez - 23102766

Victória Rodrigues Veloso - 23100460

1. Exercício 1

Para o exercício número 1, um algoritmo para realizar a multiplicação através de somas sucessivas foi implementado.

1.1 Implementação

1.1.1 Principais pontos do algoritmo

O programa inicia utilizando a instrução `jump`, e é então direcionado para a `main`. Inicialmente, os dados possuem seus endereços carregados na memória, seguido do carregamento do conteúdo desses endereços. Em seguida, os argumentos que serão passados para o procedimento são armazenados nos registradores `$a0` e `$a1`, que são direcionados ao procedimento

```
addi    $a0, $t0, 0 #armazena A
addi    $a1, $t1, 0 #armazena B
```

Em seguida, a função é chamada

```
jal     multiplicacao
```

Dentro do procedimento, iniciados um contador, responsável por controlar o loop das somas sucessivas, um registrador responsável por armazenar a soma acumulada também é iniciado.

O loop é iniciado com a soma acumulada, fazendo a operação soma = 0 + 4, seguido do incremento do contador em 1. Após as operações aritméticas, a instrução branch if not equal, compara o valor armazenado em

$a1$ (quantidade de vezes que a soma deve ser realizada) com $t2$ (contador), se forem diferentes, o loop é iniciado novamente, caso contrário o resultado da soma é armazenado no registrador de retorno

$v0$ e retornar para o ponto inicial antes do procedimento iniciar é feito através da instrução `jr $ra`

```

multiplicacao:
    li      $t2, 0    # inicia contador em 0
    li      $t3, 0    # inicia soma acumulada em 0

    loop_mult:
        add     $t3, $t3, $a0    # soma acumulada
        addi    $t2, $t2, 1      # incrementa contador

        bne     $a1, $t2, loop_mult    # se ainda não acabou a mult, volta para loop

    addi     $v0, $t3, 0    # armazena o resultado da multiplicação
    jr      $ra    # volta para o chamados

```

Após retornar para onde o procedimento foi chamado na main, o resultado é armazenado na memória e o programa finalizado.

```

jal      multiplicacao    #chama a função multiplicacao

sw       $v0, 0($s3)    #salvando o resultado na memória

```

1.2 Executando o Programa

$a0$	4	0x00000004
$a1$	5	0x00000005

Figura 1. Argumentos do procedimento

\$t2	10	0x00000000
\$t3	11	0x00000000

Figura 2. t\$2 e \$t3 iniciados

\$t2	10	0x00000001
\$t3	11	0x00000004

Figura 4. Primeiro incremento do contador e primeira soma acumulada

\$t2	10	0x00000002
\$t3	11	0x00000008

Figura 5. Segundo incremento do contador e segunda soma acumulada

\$a1	5	5
------	---	---

Figura 6. Valor do argumento (quantidade de vezes que a multiplicação deve ser feita)

\$t2	10	5
------	----	---

Figura 7. Contador com o valor igual ao argumento

\$t2	10	5
\$t3	11	20

Figura 8. Resultado da multiplicação

2. Exercício 2

Para o exercício número 2, uma função recursiva para somar elementos de uma lista foi realizado.

2.1 Implementação

2.1.1 Principais pontos do algoritmo

O programa inicia carregando o endereço de um vetor da memória no \$s1, em seguida, chama a função soma dando como parâmetro esse endereço base e o números de elementos do vetor.

A função funciona da seguinte maneira:

Confere se N (numeros de elementos do vetor) é 0, se for, a função encerra e retorna. Se não, ela adiciona o valor da posição atual do vetor (armazenado em t2) em um acumulador global de somas (t0).

Em seguida, decrementa N e muda o endereço base do vetor para o próximo, adiciona a

pilha o endereço RA e chama de novo a função recursivamente até N chegar em 0.

2.2 Executando o Programa

\$a0	4	268501000
\$a1	5	5
\$a2	6	0
\$a3	7	0
\$t0	8	0
\$t1	9	0
\$t2	10	11

Figura 1. Registradores a0 = endereço base do vetor, a1 = N, t2 = primeiro elemento do vetor

\$a0	4	268501004
\$a1	5	4

Figura 2. a1 decrementado e a0 apontando para o segundo elemento do vetor

Value (+18)
4194396
0

Figura 3. valor de retorno (ra) armazenado na pilha

\$s0	16	268500992
\$a1	17	268500996

Figura 4. s0 = endereço do resultado

Address	Value (+0)
268500992	45

Figura 5. resultado armazenado